The Unseen Block: What Split-Instant Evaluation Hides in KV-Cache Placement [Experiment, Analysis & Benchmark]

Suan Lee
Semyung University
Jecheon, Republic of Korea
suanlee@semyung.ac.kr

ABSTRACT

KV-cache placement across GPU, host, remote and storage tiers is evaluated by replaying traces, and choices such an evaluation leaves implicit decide what it reports. On two production Mooncake traces, 92.8% and 95.0% of the blocks seen before the evaluation split are never read again. A reference that knows the seen blocks' futures gives them the same belief as blocks not yet created, zero, and how an evaluation orders the two is a tie rule. With the dead blocks ordered ahead of new ones, a clairvoyant split-time reference reaches 20.4% and 15.3% of a hindsight oracle's advantage, which suggests that a split-time policy can reach little of it; with them behind new ones, the same reference reaches 77.8% and 76.2% (tier-only actions). The swing persists across splits, evaluation windows and tier capacities, and it decides whether a learned ranker is more than half of the way to the reference or a sixth. While the live blocks fit in the fastest tier, knowing which seen blocks are dead is worth as much as knowing their whole future, and a parameter-free rule that parks blocks seen once behind new ones recovers 39.2% and 43.3% (47.0% and 51.7% with an estimated prior, which three of four Alibaba serving traces support). Online, a multi-tier LRU captures 84.5% and 87.7% when moving a block is free, but at the move prices we measured on one A100 host, 0.75–1.87 times one read plus one write, and under five ways of billing a move, none of the replayed policies that move blocks beats a static frequency heuristic on the production traces in an additive cost model, even when every policy is billed for storing new blocks, unless moves to the remote tier, which we could not measure, cost under a quarter of that. Reactive replacement, S3-FIFO, SIEVE and W-TinyLFU included, stays within a few points of plain rules. An artifact regenerates every number from public traces, on CPU except the GPU timings, and we catalogue the thirty defects, four of them retracted claims, that produced plausible numbers on the way.

PVLDB Reference Format:
Suan Lee. The Unseen Block: What Split-Instant Evaluation Hides in KV-Cache Placement. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/suanlab/stoa.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

LLM serving systems now keep the attention state of past requests, the KV cache, across a hierarchy of GPU memory, host memory, remote memory and storage, so that a later request sharing a prefix reads it instead of recomputing it [2, 4, 13, 23]. Which tier holds which block is a buffer-management decision [12], and like every such decision it is evaluated by replaying a trace: let a policy observe part of it, place blocks, and bill the rest against a hindsight reference. This paper is about what that evaluation has to fix, because on production KV traces the choices it makes silently decide what it reports.

Most blocks such an evaluation shows a policy are already dead. On two production Mooncake traces, **92.8% and 95.0%** of the blocks seen before the split, the request halfway through the trace, are never read again. Dead and unseen blocks give a split-time policy no evidence of future use, and a reference that gives both the same belief orders them by a tie rule, the case LRU-K left to a "subsidiary policy" [11]. With tier-only actions, a clairvoyant split-time reference that breaks the tie by item order, putting the dead blocks first, reaches **20.4% and 15.3%** of a hindsight oracle's advantage, which suggests that a split-time policy can reach little of it; parking the dead blocks behind the new ones costs them nothing, and the same reference reaches **77.8% and 76.2%** (Section 5.2). The swing holds across splits and tier capacities, and it is not a detail: measured against the first reference, the strongest learned ranker we trained (with all actions, where it was trained) is more than half of the way there, against the second about a sixth. Neither reference is an upper bound, and an evaluation that does not state its tie rule does not say which one it reports.

Separating what the oracle knows shows what placement has to get right. On the production traces, knowing *which* seen blocks are dead is worth as much as knowing every seen block's future, because the live ones fit in the fastest tier; where they do not, as on the synthetic control, ranking them matters again (Section 5.3). The prefix approximates it: a block seen once is dead with probability 0.95 and 0.97, and a rule that parks seen-once blocks behind new ones, with no parameters, captures 39.2% and 43.3% of the advantage; an estimated prior for new blocks captures 47.0% and 51.7%, and on three of Alibaba's four serving traces it points the same way (Section 5.5). Counts mislead because KV reuse departs from the independent reference model: estimators that assume it hold on our traces once their requests are shuffled, but on the traces as recorded they overpredict a seen block's reuse 3.7 to 8.4 times, consistent with the session-bounded lifetimes Liu et al. measured [8] (Section 5.4).

Static analysis assumes that a block, once placed, stays. Replayed online, plain recency does the same job better: a multi-tier LRU

captures 84.5% and 87.7% when moving a block is free, more than any arrival-time policy (Section 5.6). Whether anything that moves blocks pays is set by the price of a move, which KV-cache studies usually assume away (Table 1). We measured it on one A100 host: a move between GPU and host memory costs 0.75–1.87 times one read plus one write, at least 2.5 times the price at which any policy that moves blocks stops paying. In an additive cost model, under every way of billing a move we tried, and with every policy also billed for storing each new block from the GPU, none of the replayed policies that move blocks beats a static frequency heuristic on the production traces, unless a move to the remote tier, which we could not measure, costs less than about a quarter of a read plus a write. The per-block costs of a serving stack (vLLM with LMCache) on the same host give the same answer, and loading a block from a cold rotational disk there costs more than recomputing it (Section 5.7). Reactive replacement, finally, stays within a few points of plain rules: the adaptive family (ARC, LeCaR, CACHEUS) beats the best plain rule chosen per operating point by at most 4.6 points of Belady, a learned LRB does not beat ARC, and S3-FIFO, SIEVE and W-TinyLFU stay below that per-point best everywhere (Section 5.9).

study	when it decides	bound reported	moves priced
CachedAttention [4]	online; warm-up on 10K turns, evaluate on the next 42K	none	storage, not moves
Mooncake [13]	online, LRU store	infinite cache	in scheduling, not eviction
KVCache in the wild [17]	online	infinite cache	"negligible"
Fancy eviction [8]	simulated online	Belady, compute-aware	compute, not moves
PolarKV [23]	online LRU, background offload	none	budget, not moves
Baleen [18]	trained on day 1, tested after	episode OPT	flash writes
this work	split instant and online replay	references with stated tie rules	measured; five ledgers

Table 1: How cache studies close to ours evaluate, from each paper's evaluation section. All evaluate online policies, whose own admission rule orders new blocks; none uses a split-time reference, the setting where the tie rule of Section 5.2 decides the result.

Contributions.

- **(C1) What an evaluation has to fix.** The decision instant, information set, billing of creations, references, tie rule for blocks without evidence, and the price and ledger of a move, each stated and swept (Section 3). The tie rule decides split-time references, which none of the online KV-cache studies we read uses (Table 1).
- **(C2) Findings on production traces**, replicated on Alibaba's traces where the data allow: the tie rule moves a clairvoyant reference by 48–63 points; while the live blocks fit in the fast tier, deadness, not ranking, is what placement has to get right, and a parameter-free prefix rule recovers about half of what knowing it is worth; at the move prices we measured, none of the replayed policies that move blocks pays unless remote moves are nearly free; modern one-hit-wonder policies do not beat the best plain rule chosen per operating point.
- **(C3) An artifact and a record.** Every number regenerates from public traces, on CPU except the GPU timings, and a checker fails on drift; the thirty defects that each produced a plausible number on the way here, four of them retracted claims, are catalogued with the control that catches each (Section 6), as a record of one project rather than an estimate of how often such defects occur.

Artifact. Every result in this paper except the GPU timings is regenerated on CPU, without a paid API, by the released package at https://github.com/suanlab/stoa; `make verify` re-derives each one from its stored result in a few minutes; `make reproduce-full` regenerates them from the public Mooncake and kv-cache-tester traces, `make reproduce-external` the Alibaba replication, and the block-move timings need a CUDA host (`measure_tier_costs.py`).

2 BACKGROUND

KV blocks are buffer-pool pages. A transformer's attention state for a token prefix is stored as fixed-size blocks keyed by a hash of the prefix, so two requests sharing a prefix share blocks. Serving systems keep these blocks across a hierarchy of GPU memory, host DRAM, remote memory and storage, and decide which tier holds which block [2, 4, 13, 23]. From a database perspective this is buffer management with a new page type [12]: pages are immutable once written, a page's first access is a compulsory miss (its KV is computed, not read), and a miss on a cached page costs a transfer rather than a recomputation. Multi-tier buffer managers have long treated the movement between tiers as a device-priced policy decision [1, 19, 24]; Spitfire, for one, migrates lazily and with tunable probabilities, and may bypass a tier, because a move has a price. Most KV-cache studies do not price moves (below); the ski-rental and cascade-aware rules of Section 5.6 are the simplest members of that lazy-migration family.

Replacement and admission. LRU-K [11] ranks pages by the time of their K-th most recent reference and states the case this paper turns on: "The only time the choice is ambiguous is when more than one page has [infinite backward K-distance]. In this case, a subsidiary policy may be used." Blocks with no usable history are that case, and on KV traces they are most blocks. 2Q [5] and its descendants admit new pages to a probationary queue; S3-FIFO [21] and SIEVE [22] demote one-hit wonders quickly; TinyLFU [3] admits a new page only if its estimated frequency beats the victim's. ARC [10], LeCaR [16], CACHEUS [14] and LRB [15] adapt or learn the replacement decision, and learning-augmented heuristics tune a simple policy's parameters instead [20]. In KV serving, HotPrefix [7] admits hot prefixes to host memory and promotes them to the GPU, and OrbitFlow [9] prices GPU–host transfers inside one request's plan.

How KV-cache placement is evaluated. Table 1 lists the closest studies. They warm a cache on a prefix of the trace and measure on the rest, or bound a policy by an infinite cache or by Belady's offline optimum. Their policies are online, so a policy's own admission rule orders the blocks it has not seen, and we do not claim that their reported numbers move with our tie rule. The tie rule

decides a different kind of result: a split-time placement analysis, or a reference for how much of an oracle's advantage a predictor could reach, which is the evaluation this paper first ran (Section 5.2). Movement between tiers is assumed free, hidden behind computation, or priced as provisioning, with Baleen's flash-write budget the exception. A concurrent study of fourteen eviction policies on production prefix-cache traces [8] finds plain LRU within 0.4 points of the best online policy at a 1 TiB pool and Belady far above all of them. It overlaps ours in three observations: block lifetimes are bounded by sessions, early frequency predicts whether a block is reused at all, and quick demotion helps only where one-hit traffic dominates. It prices recomputation, not moves between tiers, and does not vary how the measurement starts. What this paper adds is the tie rule's effect on a reference, a ladder that prices what each part of an oracle's knowledge is worth, and the price and ledger of a move.

3 WHAT AN EVALUATION HAS TO FIX

A placement evaluation fixes, often silently, five things: when the policy decides, what it may know, what it is billed for, what it is compared with, and how ties are broken. Each one moved a headline number in this study by tens of points (Section 6). This section states our choices.

Trace, blocks, tiers. A trace is a time-ordered sequence of requests, each naming the KV blocks it reads; a repeated block id is a genuine reuse because ids are content hashes of the token prefix. Blocks are immutable once computed and, within a source, uniform in size. A placement assigns each block to one of four tiers, GPU, host DRAM, remote DRAM over RDMA, and local disk, with capacities of 10%, 30%, 60% and unbounded shares of the trace's distinct blocks, counted in block slots. The experimenter sets them from the whole trace; a running system would fix them in bytes in advance, and we sweep them (Section 5.2). The replay is in slots, so a block's byte size does not enter it; the move prices we measured are for one 64 MiB block on one host and are applied per slot. A block's first access *creates* it (its KV is computed), a compulsory miss that no placement can serve, so only later accesses are billed. Billing a creation as a read from its tier, as every earlier version of this work did, inflated the oracle's advantage about threefold. Blocks are billed independently, although an engine reuses a block only behind its whole prefix. On these traces a block's id fixes its predecessor, without exception among 288,500 and 409,616 block positions, so every access to a block is an access to its prefix, and a policy whose priority grows with recency or count keeps a block's prefix at least as well placed as the block. Replayed in prefix order, LRU and Belady show no hit whose prefix missed at any tier size; we did not check the adaptive policies.

Cost. An access to a block on tier t costs $\lambda_L\, \ell_t$ plus a context-token term that does not depend on the tier, with $\lambda_L = 0.1$ per ms and per-tier read latencies ℓ_t from a declared table: 0.2, 2, 6 and 12 ms for GPU, host, remote and disk (Section 7). We report results with *tier-only* actions unless we say otherwise: a KV block is moved, not re-encoded. The artifact's cost model also lets a placement change a block's representation and when that change is paid for; we call that space *all actions* and report it only where noted, to show how much a model-dependent action space moves the numbers.

The split instant. An offline evaluation picks a split s, a request index (the midpoint unless stated), lets the policy observe the requests before s, and bills the placement on the accesses at or after s. A block is *visible* if it was accessed before s and *new* otherwise. Features, labels, the train/evaluation cut and the bill all use the same s; an earlier version of our arrival experiment used two, and leaked.

References. Every placement is billed at the true post-split cost and reported as a share of the gap between two references: the **frequency heuristic**, which places visible blocks by their prefix access count and new blocks after them, fastest tier first; and the **hindsight oracle**, which places every block by its true future count. "Captured" is $(\text{cost}_{\text{heur}} - \text{cost}_{\text{policy}})/(\text{cost}_{\text{heur}} - \text{cost}_{\text{oracle}})$. For uniform blocks, tier-only costs that rise with latency and a cost linear in the access count, the oracle's greedy order is optimal by the rearrangement inequality. No reference between the two is an upper bound for a class of causal policies here, and we do not call one a ceiling (Section 5).

Blocks without evidence. A reference that knows the futures of the visible blocks gives two kinds of block the same belief, zero: a *new* block, which no policy has seen, and a *dead* visible block, which has a history but will not be read again. A policy cannot tell a dead block from a live one; the reference can. The frequency heuristic ranks every visible block, dead or not, above every new one. Under the reference, the order of the two zero-belief kinds is set by a tie rule, usually item order. We vary that rule explicitly, and Section 5 shows it decides most of what a split-time evaluation reports.

Online replay and the price of a move. Static placements are one capacity-constrained assignment over the evaluation window. Online policies replay the post-split accesses from the same starting state, the heuristic's placement of the visible blocks, and pay for every move between tiers. We bill a move five ways (Section 5.6): one read from the source plus one write to the destination; the same with a promotion's read credited to the serving access that caused it; with clean lower copies kept, so demoting onto one is free; with every block written through to host memory at creation; and free. We also measured what a move costs on one GPU host. The bill is additive: every read and every move adds its price, with no overlap, queueing or batching, and every block stays in some tier, with no discard-and-recompute action.

4 WORKLOADS AND SETUP

We use **Mooncake** [13], replayed production traces from a general LLM serving stack (conversation, 12,031 requests; toolagent, 23,608; 512-token blocks), and its third released workload, synthetic (public datasets, Poisson arrivals), as a control rather than pooled. Alibaba's four Qwen serving traces [17] (16-token blocks, full length) are an external replication, and **kv-cache-tester**, anonymized agentic coding conversations (64-token blocks), a second source for reactive caching. Each request lists the hash ids of the blocks it reads, so a repeated hash is a genuine reuse. Block ids in kv-cache-tester are

	Mooncake (serving)	kv-cache-tester (agentic)
requests analysed	12,031 / 23,608	4,215 (40 conv.)
blocks touched exactly once	75.8 / 78.5%	15%
mean reuse per block	1.6 / 2.2	26
blocks per request	24 / 17	~1,900
block size (tokens)	512	64
headroom, heuristic vs. oracle	93%	68–73%

Table 2: The two sources, at the scales actually analysed. Mooncake pairs are conversation / toolagent; its headroom averages the two (93.5% and 91.8%). The agentic column is the first 40 conversation files, except headroom, which is the mean over four randomized draws of 20 and of 40 conversations (Section 5.10). Its descriptive statistics are sample-dependent: the one-time fraction ranges 7.5–21% over 20–200 conversations. An earlier revision took this column's headroom from nested-prefix samples (81–94%), the protocol Section 5.10 retracts.

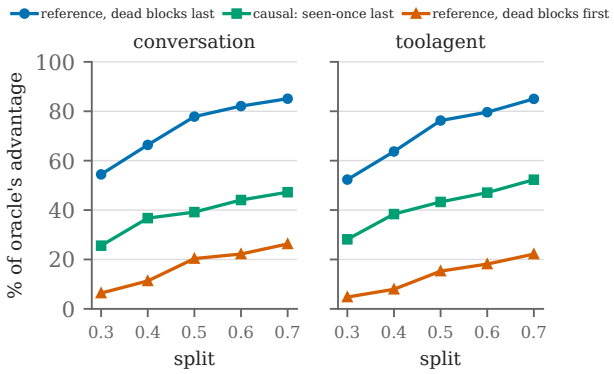


Figure 1: The same clairvoyant split-time reference under the two tie rules for blocks without evidence, and the causal rule that parks seen-once blocks behind new ones, across splits (tier-only actions). The tie rule moves the reference by 48–63 points.

conversation-scoped, so we namespace them before pooling; not doing so invents cross-conversation reuse. Table 2 summarizes the two primary sources. The third-party traces are not redistributed; the artifact downloads them and checks pinned SHA-256 digests.

Every static and replayed result is a deterministic CPU computation. Block moves were timed on one host with an NVIDIA A100 80GB on PCIe Gen4 x16, an AMD EPYC 7513 host, and a rotational RAID-backed disk, on an otherwise idle GPU while the host's other GPUs ran other jobs; six runs of a 64 MiB block (an 8B-class model with grouped-query attention, 512 tokens). The host has no NVMe and no RDMA fabric.

5 BLOCKS WITHOUT EVIDENCE

5.1 Most blocks a policy can see are dead

At the midpoint split, **92.8% and 95.0%** of the blocks visible on the conversation and toolagent traces are never read again, and 88.8–96.0% across splits from 0.3 to 0.7 (64.4% on the synthetic control). The few that are read again, 7,248 and 4,901 blocks, carry 45.7% and 74.5% of the billable post-split reuse, and new blocks the rest, 54.3% and 25.4%. A policy that places blocks by their prefix counts therefore fills the fast tiers mostly with blocks that will not be used, ahead of the blocks it has not seen yet. A policy cannot tell the dead blocks from the live ones; what it sees is a count that will not grow, or for a new block no count at all. A reference that knows the visible futures gives dead and new blocks the same belief, zero.

5.2 The reference is set by a tie rule

A natural reference for split-time policies gives every visible block its true future count and every new block zero, and places them greedily (Section 3). Its dead visible blocks also get zero, and which of two zero-belief blocks goes first is a tie rule. A stable sort in item order puts every visible block ahead of every new one, and the reference then reaches 20.4% and 15.3% of the oracle's advantage with tier-only actions (15.6% and 12.4% with all actions), which reads as most of the advantage being out of a split-time policy's reach, the conclusion an earlier version of this paper drew and that we retract (Section 6). Parking the dead blocks behind the new ones instead costs them nothing — they are never read — and the same reference reaches:

reference (tier-only actions)	conv.	tool.	synth.
dead visible blocks ahead of new ones	20.4	15.3	60.2
dead visible blocks behind new ones	**77.8**	**76.2**	84.1
same, all actions	15.6 / 50.8	12.4 / 49.4	45.4 / 62.7

Percent of the oracle's advantage over the frequency heuristic, split 0.5.

With tier-only actions the rule moves the reference by 48–63 points across splits on the production traces, and by 10–72 points when the three bounded tiers are scaled together from a tenth to one and a half times their sizes; the swing is smallest at the smallest tiers, where the reference with dead blocks last reaches only 42–48% (the synthetic control moves by 0–40 points). In serving units, under the declared latencies the oracle's advantage is 32.6 and 16.2 ms of block reads per request after the split, so the rule moves the reference by 18.7 and 9.9 ms per request.

The rule decides how a result reads. The strongest ranker of Section 5.8, an MLP trained with all actions, gains 8.2% and 7.4% of the oracle's advantage: 52.6% and 59.5% of the way to the reference with dead blocks first, a learned model past halfway to what looks reachable, but 16.2% and 14.9% of the way to the reference with them last.

Neither reference is an upper bound for the policies it stands for: on 200 random ten-block workloads an exhaustive search over placements of the visible blocks beats the dead-first reference in 90 and the dead-last one in 83, by reserving fast slots for arrivals, which needs knowledge of the arrivals. Knowing only the arrivals' average does not lift the reference on the production traces: parking every visible block whose future is below the mean future of the new blocks (0.35 and 0.34 accesses) parks exactly the dead ones there, so the dead-last reference is already the one a split-time policy can build from the arrival distribution. On the small workloads the average does help: the same rule closes 52.4% of the gap between

the dead-last reference and the exhaustive search. We therefore report both references and call neither a ceiling.

5.3 Knowing which seen blocks are dead is most of the prize

Separating what the oracle knows into parts, each billed at the true cost (tier-only, split 0.5; the last three rows are clairvoyant):

	conv.	tool.	synth.
frequency heuristic	0.0	0.0	0.0
causal: seen-once behind new	39.2	43.3	15.4
causal: estimated prior	47.0	51.7	0.0
which visible blocks are dead	77.8	76.2	67.0
visible futures, dead last	77.8	76.2	84.1
oracle	100	100	100

On the production traces, knowing only *which* visible blocks are dead is worth as much as knowing every visible block's future, to the hundredth of a point. That is a regime, not a law: the 7,248 and 4,901 live visible blocks fit in the GPU tier's 18,279 and 18,330 slots, so their order cannot matter. Shrinking the tiers until up to four times as many live blocks as GPU slots remain opens a gap of at most 1.9 points; on the synthetic control, with 1.8–27 times as many, ranking the live blocks is worth 17–70 points. What remains, 22–24%, is the order among new blocks. The prefix approximates deadness well: a block seen once before the split is dead with probability 0.95 and 0.97, and seen-once blocks are 78–80% of the dead ones. Parking them behind new blocks, a rule with no parameters, captures 39.2% and 43.3% (25.6–52.3% across splits). Liu et al. observe that early frequency predicts whether a block is reused at all [8]; the ladder prices that observation inside a placement. The estimated prior of Section 5.5, which places new blocks between seen-twice and seen-three-times, captures 47.0% and 51.7%. On the synthetic control fewer blocks are dead (57–72%), seen once is a weaker sign of it, and the same rule captures 7.5–15.4%. The rule also needs room: with the bounded tiers at a quarter of their sizes it captures 14.1% and 16.4% on the production traces, at a tenth nothing.

The static assignment is realized by one rearrangement at the split, which the static bill does not charge, and by admitting each new block on creation into the tier the assignment reserved for it. Charging both at the measured move prices (Section 5.6), the seen-once rule loses 37–160% against the heuristic: what the ladder measures is the value of knowing which blocks are dead, not of moving blocks after the fact to act on it.

The pattern replicates on Alibaba's four Qwen serving traces [17] (16-token blocks, split 0.5, tier-only): 87.5–95.8% of visible blocks are dead, the reference moves from 2.3–18.3% with dead blocks first to 53.7–76.5% with them last, knowing deadness again equals knowing every visible future, and the seen-once rule captures 25.8–34.2%, including on the trace where the estimated prior of Section 5.5 gains nothing.

The direction a block's evidence points — toward reuse or away from it — is thus most of what a split-time placement has to get right, and it is a question about blocks with almost no evidence.

5.4 Why: KV reuse departs from the independent reference model

Buffer-management theory from the A0 policy to LRU-K [11] often models each page as referenced independently at a constant rate, the independent reference model. Under it two estimators need nothing but the counts: Robbins' identity predicts the mean future count of a block seen k times from the number of blocks seen k and k+1 times, and Good–Turing's missing mass predicts the share of accesses that go to blocks not yet seen. Stationarity alone does not give either: a stationary stream whose pages are referenced in bursts fails Robbins' identity too, so what the test below rejects is the independent reference model. With the requests shuffled into random order, the ratio of actual to predicted reuse for blocks seen once is 1.04 and 1.05, 0.86–1.31 over twenty shuffles, and the missing mass matches the share of accesses to new blocks within 0.04; for blocks seen three or more times too few blocks remain for the identity to be tight (0.16–4.0). On the traces as recorded both fail. Blocks seen one to four times are reused 0.12–0.27 times as often as the identity predicts (0.51 and 0.65 for blocks seen five times), and new blocks receive 82.2% and 57.4% of the accesses after the split where the missing mass predicts 50.1% and 36.4%. Three of Alibaba's four traces depart the same way (0.19–0.31 for blocks seen once); the fourth, traceB, is close to the model (0.71). The size of the departure does not predict what the estimated prior gains: it gains 34.2% on traceB and nothing on thinking, the trace that departs most. The synthetic control departs the other way (1.95), for reasons we did not examine.

The departure is what one expects if a KV block is created with a session and stops being read when the session ends, so that its count says more about how long it has been alive than about how often it will be read; Liu et al. measure such session-bounded lifetimes directly on other production traces [8]. A full shuffle cannot tell sessions from drift over the trace, so we also shuffled chunks of consecutive requests, keeping the order inside each chunk.[1] The criterion we registered read the departure as local structure if blocks seen once stayed below 0.5 with 1,000-request chunks and as drift if they reached 0.8. They reach 0.75 and 0.85 (0.84 and 1.10 with 100-request chunks, 0.51 and 0.53 with 3,000), so the departure is not local: it needs the trace's order kept over thousands of requests, minutes rather than seconds, and the test does not separate long sessions from drift.

Two consequences follow, and we tested both. If blocks die with sessions, they die quickly, and the dead share should not depend much on how long the evaluation waits. It does not: with the split at a quarter of the trace, cutting the window after it from about 42 minutes to 7.5 moves the dead share from 87.5% to 90.2% on conversation and from 90.5% to 91.4% on toolagent, by 1.8 and 0.2 points between 15 minutes and the end. The window's length is nonetheless one more convention for the reference: as it lengthens, more of the oracle's advantage falls on blocks created inside it, and the reference with dead blocks last falls from 89.7% to 47.5% on conversation, while the tie rule still moves it by 38–52 points at every length. Those capacities follow each window's blocks, so the

[1] A registered rule or criterion is written, before the first run, into the docstring of the script that tests it. Script and result were committed together, so the repository shows that the criterion has not changed since, not that it preceded the run.

system changes with the window. Holding the tiers' bytes at the full trace's, the reference still falls, from 90.2% to 47.5% and from 90.6% to 45.5%, and the tie rule moves it by 40–56 points; the seen-once rule, however, depends on window and tier size together, and with tiers sized for the full trace it loses 16.5% and 12.4% at 7.5 minutes. Second, a block's age should mark it as dead better than its count. It does, but that is not enough: the rule we registered in advance, parking blocks seen at most twice and older than the median, parks blocks that are dead 98.0% and 99.9% of the time but reaches only half of the dead ones, and captures 28.0% and 28.8%, less than the seen-once rule. Parking the oldest three quarters, which we tried afterwards, captures 49.0% and 50.4%, but on Alibaba's traces it beats the seen-once rule on two, falls below it on the other two, and loses 19.1% on traceB, the nearly stationary trace, where age says least; the registered rule is below the seen-once rule on all four. Every rule we tried gains on every trace except that one, and none beats the seen-once rule on every trace. Measuring age by the session instead of the block, with the session taken from the prefix as the block after the first branching point of its hash chain, changes the result by less than a point (29.8% and 30.6% against 30.5% and 31.1%), since reading a block reads its prefix. With every rule's precision at 0.95 or more to two decimals, what decides a rule's value is how many of the dead blocks it reaches.

5.5 Deciding at arrival

A policy need not wait for a split: it can rank each block when it is created, against the blocks already placed. The decision that matters is the same one: where a block with no history ranks against a block seen once. Every block in a static analysis gets a belief that reads nothing after its own creation; beliefs are placed in one capacity-constrained assignment. Ranking new blocks among themselves (by arrival order, by a learned score from session context, or at random) moves the result by at most 5.3 points; moving them ahead of or behind equal-count visible blocks moves it by 14–29 points. An *estimated prior* replaces that tie rule: at a pseudo-split halfway through the prefix, blocks born after it are reused before the real split more than blocks already seen once (0.52 against 0.30 on conversation), so new blocks are placed between count 2 and count 3. It captures 47.0% and 51.7% with tier-only actions (58.2% and 61.1% with all actions), and nothing on the synthetic control with tier-only actions. On Alibaba's four Qwen serving traces [17] (16-token blocks) it captures 54.3%, 34.2% and 45.9% on three and 0.0% on the fourth (thinking), where the prefix rates seen-once blocks above new ones. The direction the prefix gives is a regularity of these workloads, not a law, and the estimate has to be checked per workload.

5.6 Online: recency, and the price of a move

Static analysis assumes a block, once placed, stays. Replayed online from the heuristic's starting state with tier-only actions, the same arrival-time policies capture up to 57.3% when moving a placed block is free. Plain recency does better on the same bill: a multi-tier LRU that moves every accessed block to the fastest tier captures **84.5% and 87.7%**; caching a block only on its second access captures −6.2% and −2.8%. Online shares keep the static denominator, the static oracle's advantage over the heuristic, so they compare policies

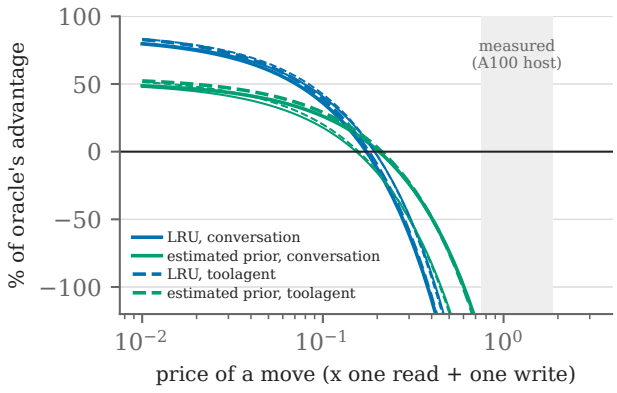


Figure 2: Online replay: captured share against the price of a move, as a multiple of one read plus one write, for multi-tier LRU and the estimated prior. Thick lines bill every move in full; thin lines write every block through to host memory at creation. The shaded band is the range we measured.

on one scale; an online policy is not bounded by a static oracle, and its share could exceed 100%. Creations are unbilled for every policy, online or static.

Whether either pays is set by the price of a move. Billed one read plus one write per move, LRU falls to −400.5% and −401.4%, no replayed arrival-time policy beats the static heuristic, and the estimated prior keeps 70% of its static gain only if a move costs at most 7.3% and 7.2% of that price. We timed the price on one host (an A100 80GB on PCIe Gen4 x16, six runs, a 64 MiB Mooncake block of an 8B-class model): a move between GPU and host memory costs 0.75–1.87 times the read-plus-write price, whether it blocks serving or overlaps it, because the two copy directions did not overlap on this host and a background move took the link time of a load (timed with the link saturated; a stack whose link sits idle between loads could hide part of a move, which this additive bill does not represent); a demotion to the host's rotational disk costs 0.90–1.46 times. Every measured price is at least 10 times that 70% threshold.

The conclusion does not depend on how the hierarchy bills a move. Crediting the serving read to a promotion, keeping clean lower copies of immutable blocks, or writing every block through to host memory at creation (the last two without charging the copies' space) leaves LRU at −401.4% to −353.2%, and none of these policies pays on the production traces once a move costs more than 0.28 of the read-plus-write price (the price at which its share reaches zero); most of LRU's moves demote blocks just created, which have no copy below. Only on the synthetic control does a moving policy pay, cache-on-second-hit under the two copy-keeping ledgers (2.6% and 4.5%). The remote tier, which receives most of the arrival-time policies' online demotions, could not be measured.

One ledger for static and online. Two asymmetries remain in these bills. A block created after the split costs nothing wherever it lands, although its KV is computed on the GPU, so a static placement that puts it on the remote tier performs a store that LRU, which creates on the GPU, never makes; and a static assignment's rearrangement

at the split is free. Billing every policy for both, each new block's store from the GPU and each static policy's rearrangement, at one price in the measured band and on the same scale as above, LRU still loses 46–333% on the production traces under every ledger and caching on the second hit 275–892%, while even the oracle's placement keeps at most 65% of its advantage. Pricing the paths separately bounds the tier we could not measure: with GPU–host moves at 0.75 and every move that touches the remote tier or the disk free, LRU gains 20.4% and 20.8% under write-through. It stops paying once such a move costs more than 0.02–0.23 of read-plus-write, depending on the ledger, and never pays under the reread ledger or with GPU–host moves at 1.87. The conclusion therefore rests on a remote move costing at least about a quarter of a read plus a write, which we did not measure.

Moving a block only once it has paid for the move does not change this. A ski-rental policy promotes a block when the extra cost of its accesses from a slower tier reaches the price of the move, the rule that is 2-competitive for one block's rent-or-buy decision [6]. It breaks even at 0.27 and 0.30 of the read-plus-write price, later than LRU (0.17 and 0.18), but at every price in the measured band it loses 40.7–97.2%; even its 0.30 is less than half of the lowest price we measured. Each promotion into a full GPU tier forces 1.94 and 1.95 demotions on average, 62% of its move cost, and the rent-or-buy rule does not see them. Counting that cascade in the threshold, which we tried afterwards, cuts the loss to 4.1–16.8% but not to zero. The break-even price can be estimated from the past: replaying a policy on the trace before the split puts it 19 to 33% below the value observed after the split, low in every case.

5.7 What a serving stack measures

We ran an 8B-class model with the Llama-3-8B attention shape (DeepSeek-R1-Distill-Llama-8B) in vLLM 0.11 with LMCache 0.3.9 on the same A100 host, and timed the first token for a prefix held in each place:

prefix tokens	recompute	GPU cache	host memory	disk (cold)
512	50.4	13.8	17.1	58.7
2,048	157.9	16.3	26.7	201.1
8,192	678.5	23.2	64.2	749.6

Median time to first token (ms) over seven fresh prefixes each, in one process per column, one request at a time; the host's rotational RAID disk read after its page cache and controller cache were flushed.

Per 512-token block, loading from host memory costs 2.56 ms and storing there 2.94 ms; recomputing costs 41.0 ms; loading from the cold disk costs 45.4 ms, more than recomputing, so on this host a disk tier does not pay at any prefix length we tried. A first run that did not flush the disk controller's cache loaded twice as fast as the array reads cold; we report it as cache-assisted and do not use it. Under these costs, moving a block from GPU to host memory costs 0.77 of the read-plus-write price. These are per-block costs from a tier probe, a block's cost taken as linear in the prefix length; we did not replay the policies inside the serving stack, so what follows is the additive model under measured costs, not a measurement of the policies' latency or throughput. Replacing the declared latency table with the measured costs, and sweeping the unmeasured remote tier from twice the host cost to just under the disk cost, leaves every conclusion in place: the reference moves from 13.8–21.9% with dead blocks first to 69.1–89.7% with them last, the seen-once rule captures 31.8–58.9%, and LRU, at 82.2–92.3% with free moves, falls to −574.8% to −220.3% at the read-plus-write price and breaks even only below 0.13–0.29 of it. Every replay keeps each block in some tier; none may drop a block and recompute it, although on this host that is cheaper than a cold-disk load, so a policy allowed to do so might do better than the placements compared here.

5.8 What a better ranker buys

Does a better model of the visible blocks pay inside a split-time protocol? We train a ladder of rankers on one production trace and evaluate on the other, with every rung's scores quantile-matched onto one magnitude multiset so that only the order varies. Placement improves monotonically with ranking quality, from the frequency heuristic, which log(count) reproduces exactly, to an MLP with cross-trace AUC 0.82 and 0.93, but with all actions, the space the ladder was trained in, the strongest model gains 8.2% and 7.4% of the oracle's advantage, a third to a half of what the parameter-free seen-once rule gains in the same space by moving blocks without evidence behind new ones (17.9% and 23.1%; Section 5.3). The magnitude attached to an identical order moves the result more than the order does: running each rung on its own score scale and on the common multiset differs by **22 to 381 points** of the reference, because with all actions a placement reads magnitudes as well as order: a block's belief decides which representation it can afford, not only its rank. With tier-only actions the fastest tier with room is always the cheapest, and only the order counts. A "fraction captured" is not a quantity until its magnitude convention is stated.

5.9 Where reaction stands

The alternative to forecasting is to react. On the full Mooncake traces, reactive tiering reaches 75–99% of Belady's hit rate on the toolagent trace and 40–94% on open-ended conversation using LRU, LFU or a fill-once cache alone; admitting the adaptive and learned policies below widens the per-operating-point envelope only to 76–99% and 42–94% (Figure 3). This section's metric is a single cache's hit rate as a fraction of Belady's, with no move price, not the captured share of the sections before it.

The interesting question is not the envelope — that is an oracle over policies, not something anyone deploys — but whether any *single* policy attains it, and whether the adaptive designs built for exactly this situation beat plain rules. We therefore run the family: **ARC** [10], which shifts a recency/frequency split on ghost hits; **LeCaR** [16] and **CACHEUS** [14], which replace ARC's arithmetic update with regret-based multiplicative weights over two experts; and the learned **LRB**, which is reported separately below because its numbers carry an implementation band the others do not.

trace	tier	best pure	ARC	LeCaR	CACHEUS
conversation	2%	39.8	29.4	**42.3**	35.9
	5%	53.3	**57.9**	56.8	53.5
	10%	76.0	**77.5**	77.2	75.3
	20%	**94.1**	87.4	93.9	88.8
toolagent	2%	74.9	70.7	**75.8**	73.9
	5%	81.3	**83.6**	83.0	82.9
	10%	91.9	92.3	**92.9**	91.8
	20%	99.3	96.3	**99.3**	97.6

Percent of Belady's hit rate. "Best pure" is the best of LRU, LFU and a fill-once cache (which admits blocks until it is full and never evicts) at that operating point, itself a choice among policies, so the comparison is conservative for the adaptive designs.

Across eight operating points and four designs, **the best adaptive or learned policy beats the best pure rule by at most 4.6 points of Belady**, and at the loosest tier by at most 0.1. Which design wins also moves: LeCaR at the tightest tier on both traces and ARC at 5%; above that, ARC and then plain LRU on conversation, LeCaR on toolagent. That is the substantive finding — not that these policies fail, but that on KV-block workloads the whole adaptive-replacement family collapses into a band a few points wide, well short of the oracle at the small tiers where the decision actually matters.

The diagnostics suggest why. Every policy in this family adapts on a *ghost hit* (a miss on a block it recently evicted), whether it updates a split (ARC) or an expert weight (LeCaR, CACHEUS). But 76–78% of blocks in these traces are never reused at all, so most evictions are correct and produce no signal. Measured, ARC receives an adaptation signal on **3.5–7.3% of misses**; the regret-based policies read the same channel, which is why they land in the same band rather than above it. What signal it does get is not merely sparse but, at the smallest tier, directionally wrong: at a 2% tier ARC drives p to 90% recency on both traces, while the rule that actually wins there is frequency. A workload whose blocks are mostly singletons starves the very mechanism adaptive replacement runs on.

The designs built for one-hit wonders do no better. S3-FIFO [21], SIEVE [22] and W-TinyLFU [3], reimplemented from their papers (W-TinyLFU with exact, periodically halved counts in place of its sketch), stay below the best pure rule at all eight operating points, by 0.1 to 8.9 points of Belady; the closest is W-TinyLFU at the tightest tier. Quick demotion does what it was designed for: at the tightest tier S3-FIFO and W-TinyLFU beat LRU and ARC on both traces. What they do not beat is the best plain rule chosen per operating point, and we did not isolate why.

And the learned cache policy loses to both — but only as a range. The natural learned baseline is LRB [15], which learns a relaxed-Belady objective from a far richer feature set than our four trace-local statistics and won on production CDN traces. We reimplement it: GBDT regression on log time-to-next-request over inter-access deltas and exponentially decayed counters, sampled eviction, online retraining from a sliding memory window, right-censored labels.

Our first version of it measured something else entirely. The training buffer was capped by truncating to its tail; labelled rows arrive one at a time as blocks are re-accessed while censored rows arrive in one contiguous burst per checkpoint, so from the third retrain onward every training label was identical, the regressor returned a constant, and eviction degenerated to a uniform draw over the sampled candidates. We report the repaired numbers and, because two defensible repairs disagree, we report them as a band:

trace	tier	LRB (repaired)	ARC	below ARC?
conversation	2%	27.6–29.2	29.4	yes
	5%	30.9–32.2	57.9	yes
	10%	45.0–45.9	77.5	yes
	20%	75.8–76.5	87.4	yes
toolagent	2%	68.8–69.0	70.7	yes
	5%	71.1–71.4	83.6	yes
	10%	80.9–83.0	92.3	yes
	20%	94.6–94.9	96.3	yes

Percent of Belady, seed 0. The band spans two repairs of the truncation bug: subsampling all accumulated rows, and dropping rows older than the memory window first. We adopt the second, which is what a "sliding memory window" means.

What the band supports. The two repairs agree to within 2.1 points at every operating point, and with seed 0 LRB is below ARC at all eight operating points under both. LRB samples its eviction candidates and training rows at random, so we reran both repairs with ten seeds at the 2% tier, where the margins are smallest. The criterion we registered kept the ordering only if no seed reached ARC; in three of the four trace–repair pairs one or two of ten seeds reach or pass it there, by at most 0.3 points of Belady, and the seeds spread LRB over up to 6.4 points on conversation. At that tier LRB and ARC are indistinguishable. The claim we keep is narrower: **LRB does not beat ARC**, and it is below ARC by 9 points or more at five of the other six operating points, more than the seed spread; at toolagent's 20% tier the margin is 1.4 points, inside it. Both repairs and the retracted behaviour are selectable in the artifact, so the band is regenerated rather than reported. We do not claim the band bounds every reasonable reimplementation: a third repair produced in an AI-agent review we ran (see the Acknowledgments) reported a figure at the 10% conversation tier above our band but still below ARC; since its code is not ours to ship, we neither reproduce nor rely on it.

The mechanism appears to be the one that limits ARC, seen from the other side. LRB learns from labels the trace supplies when a sampled block is requested again; a block that never returns yields a *right-censored* row, which says only "not within the window" and nothing about when. At the default window **83–88% of LRB's training rows are censored**, and 88–91% at a shorter one. A population dominated by singletons starves the ghost-hit channel that adaptive replacement runs on and the regression target that learned replacement runs on, for the same underlying reason.

Three caveats bound this. It is our reimplementation of the published mechanism, not the authors' artifact. KV blocks are uniform-size, so byte miss ratio collapses to object miss ratio and LRB loses one of its structural advantages over LRU. And our first hyperparameter sweep ran on the *broken* implementation, so it tested the robustness of random eviction. Rerun on the repaired one at a 10% tier (six settings of window, sample size and trees per trace), LRB spans 41.9–61.8% of Belady on conversation and 77.6–86.4% on toolagent, below ARC's 77.5% and 92.3% at every setting; the band is too wide to quote one LRB number, but not wide enough to reach ARC.

The eviction signal also inverts by source. On the agentic kv-cache-tester traces LRU reaches 0.83–0.96 of Belady at a 5% tier while LFU stays near 0.29–0.31; on Mooncake the ordering reverses below the working-set knee. An eviction policy tuned on one source

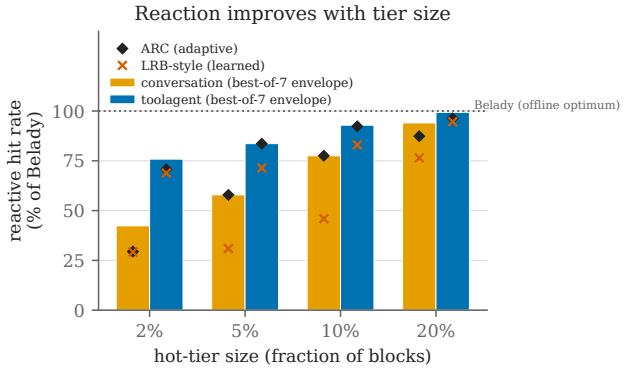


Figure 3: Reactive tiering as a fraction of Belady's hit rate, full Mooncake traces. Bars are a per-operating-point best-of-seven envelope, an oracle over policies rather than a policy. Diamonds are ARC, which meets the envelope at 5% on both traces and at 10% on conversation but falls short elsewhere; crosses are the LRB-style learned policy (seed 0), which is below ARC everywhere; other seeds reach ARC at the 2% tier.

	sorted-prefix samples (as first reported)	4 random draws mean [observed range]
20 conversations	—	−338% [−858, +78]
40 conversations	+88%	−44% [−166, +66]
70 conversations	+75%	—
100 conversations	−9%	—
150 conversations	−48%	—

Table 3: Headroom captured by a learned placement policy on agentic traces, the same pipeline measured two ways. Left: sorted-prefix samples, which are nested and drawn from a corpus whose per-conversation statistics vary threefold along the file index — the numbers we first reported (regression variant, all features), and their reversal; an earlier revision printed the classifier's +87% at 70. Right: randomized draws at the same sizes, summarized by their observed range — four draws cannot support a parametric interval. The spread is an order of magnitude wider than any effect, so this quantity was never measurable under this protocol. We report it as a cautionary measurement.

is a poor default for the other — and, as both the ARC and the LRB result show, reaching for a more sophisticated policy does not by itself resolve the choice.

5.10 A measurement that was never a measurement

We also attempted the obvious follow-up (fit a richer per-block model on the agentic source and place with it) and first obtained what looked like a strong positive: +66 to +88% of the headroom captured, reproducing across what we took to be two samples. It was not a result, and the way it failed is worth more than the number would have been.

The two samples were *sorted prefixes* of the conversation corpus, so the smaller was contained in the larger; and file index correlates strongly with conversation shape (mean requests per conversation falls from 140 to 37 between files 1–20 and 71–150). Extending the same prefix to 100 and 150 conversations drove the captured headroom to −9% and −48% — a sign reversal. Re-drawing the samples *at random* rather than as prefixes then showed why: four random draws at 40 conversations land between −166 and +66 points, and at 20 conversations between −858 and +78 (Table 3). The spread dwarfs the effect by an order of magnitude. We quote the observed range rather than a confidence interval on purpose: with four draws a parametric interval rests on a normality assumption these data cannot support, and *captured* is a ratio whose denominator varies per draw. An earlier revision of this paper reported ±126 here, computed with the normal quantile against a standard deviation estimated from four points; the correct $t(3)$ half-width is ±166 on the current artifact. That error was ours. Under a correct sampling protocol there was never a measurement here to report.

We keep this in the paper for two reasons. It is the control that our own protocol (Section 6) lacked, and running it costs minutes; and it bounds what the rest of our numbers are worth, since every other result in this section is also a single draw of a parameter we chose.

6 WHAT THE MEASUREMENT TOOK

Thirty defects produced plausible numbers that matched their artifacts on the way here (Table 4), and four of them were claims we had to retract: the three in the row of that name and the tie rule. Most were found by re-verification passes after the paper passed every check it defined for itself; two by an adversarial review and five by an audit, both run with AI agents (see the Acknowledgments), one by a later review that asked whether our reference was an upper bound, and the last two by later AI-agent reviews. The catalogue records one project; it is not an estimate of how often such defects occur elsewhere. The artifact's lab notebook records each with its fix.

Three lessons generalize beyond KV caches. *State the order of blocks without evidence*: any reference or policy that gives two blocks the same belief places them by a tie rule, and when most blocks have no evidence the rule is the result; LRU-K named this case in 1993 and left it to a "subsidiary policy" [11]. *Treat every convenience choice as a variable*: the split, trace length, action space, tie rule, magnitude scale and billing of creations each moved a headline by tens of points here. *Price what a policy does, not only what it serves*: a policy that moves blocks is only as good as the move is cheap, and the price has to be measured or bounded rather than assumed. The harness ships with these as sweeps, and with a checker that re-derives every number in this paper from its artifact and fails on drift.

7 LIMITATIONS

Costs were measured on one host, with one model and one serving stack (Section 5.7); its disk is rotational and it has no NVMe and no RDMA fabric, so the remote tier's cost is swept rather than measured, and the online conclusion holds only if a remote move

mechanism (count)	what happened	control now in the harness
Reference-frame mismatch (6)	a split over access-event indices that fell past the trace's end; ties broken by label (AUC 0.51 to 0.94); an oracle inheriting the heuristic's order; features built at one split instant, costs billed at another	one split instant for features, labels and bill; frames compared explicitly
Retracted claims (3)	retracted: a "+66–88% captured" agentic headline that sampling showed was never measurable; the *decoupling* claim, an artifact of an unreachable denominator; an LRB column that measured random eviction because its training buffer was truncated	randomized draws; references reported under both tie rules; LRB fails on zero label variance
Free-parameter extrapolation (3)	curves read from one point: a cache size, a trace length, a conversation count (+88% reversing to −48%)	every convenience parameter swept
Believing a report (1)	an edit script printed success after matching nothing	edits asserted to land
Stale derived or parallel surfaces (3)	artifacts older than the code that made them; figures fifteen days older than their data; a notebook stating a falsified claim	content-hash provenance; figure and PDF manifests; banners on records
Unchecked summaries (3)	a sentence misquoted a checked band's width; a ratio set a ranker's gain with all actions against a rule's tier-only gain ("about a fifth", in fact a third to a half); a sentence said the estimated prior gains least on the trace closest to the independent reference model, where another trace's artifact shows no gain at all	spans and maxima asserted; withdrawn values must be absent; ratios computed in one action space
Unverified interventions (1)	a mutation whose target string was absent "passed"	interventions asserted to apply; guards tested both ways
A tie rule in the reference, retracted (1)	the split-time reference broke the zero-belief tie by item order, putting dead visible blocks ahead of new ones: 20.4% and 15.3% of the advantage, 77.8% and 76.2% the other way (tier-only); retracted: "most of the advantage is out of a split-time policy's reach"	both rules reported; an exhaustive check on small workloads
Emphasis without verification (1)	the headline share sat in no artifact and mixed two references (97% to 95%, then 84–88%)	every emphasised number asserted
Leaks and unstated conventions (8)	in the arrival experiment: post-split labels; a cut unlike the bill; magnitudes from evaluation blocks' futures; zero beliefs for pre-split blocks; a mixed-scale ceiling; an order control that also changed magnitude; unstated tie and action-space conventions (up to 56 points); creations billed as reads (the gap inflated about three times)	a causal module with an AST taint test; conventions swept; billable access defined in one place

Table 4: The thirty measurement defects, by mechanism. Each produced a plausible number that matched its artifact.

costs at least about a quarter of a read plus a write (Section 5.6). The bill is additive and per block: it has no overlap of moves with computation or idle link time, no queueing or batching, and no discard-and-recompute action, and we did not replay the policies inside a serving stack, so none of our results is a statement about latency or throughput under load. The replay counts tiers in block slots: at the 64 MiB block we timed, the GPU tier's 18,279 slots would be a 1.1 TiB pool, and applying one host's per-block price to a pool of that size assumes that transfers scale with blocks. The results use a declared per-tier latency table and are rerun under the measured one; its GPU:CPU latency ratio is 1:10 where five derived device classes give 1:80 to 1:250, except a deliberately flat one, a raw copy on our A100 host measures 1:58, and the serving stack, where a GPU-resident block still costs attention time, about 1:5. Deriving the table from three attention shapes and five device classes puts the headroom, the share of the heuristic's cost the oracle saves, at 55–98%, which contains the 92% the declared table gives in the same run (Table 2 averages two traces to 93%), but we did not rerun the ladder or the online replays under the derived tables, and 6 of the 15 configurations reorder the hierarchy in a way our fixed capacities cannot express.

Every placement result is a single deterministic run with no confidence interval, though the split, trace length, evaluation window, tier capacities, cost weights, tie rule, action space and billing are swept rather than fixed; the randomized controls are repeated (twenty request shuffles, ten LRB seeds at the tightest tier; LeCaR and CACHEUS, also randomized, are single runs), and the ladder, the stationarity test and the age rules are replicated on Alibaba's four traces. The sources are KV-block hash streams from transformer prefix caching: independent in provenance, not in mechanism, and we cannot separate workload character from vendor, date or block size. The estimated prior fixes one pseudo-split, its count bins and its age quartiles; rescaling for its half-prefix counts moves it by at most 0.6 points on the production traces. Our LRB, LeCaR, CACHEUS, S3-FIFO, SIEVE and W-TinyLFU are rebuilt from their papers, not the authors' artifacts (CACHEUS's CR-LFU expert approximated by LFU, W-TinyLFU's sketch by exact aged counts), and uniform block sizes remove one of LRB's advantages over LRU, so their results are statements about this workload rather than about these designs. The best plain rule they are compared with is chosen per operating point in hindsight; we did not test choosing it from the prefix. Hawkeye and Mockingjay are not run: a KV block hash carries no analogue of the program counter their predictors are indexed by.

8 CONCLUSION

On production KV-cache traces, nearly every block a placement policy can see at a split is never read again; the few that are fit in the fast tiers, and the rest of the reuse falls on blocks it has not seen. How an evaluation orders these blocks without evidence decides what it reports: the same clairvoyant split-time reference reaches 15–20% of the oracle's advantage or 76–78% (tier-only), depending on a tie rule that a split-time evaluation has to state and that LRU-K left to a subsidiary policy in 1993. While the live blocks fit in the fastest tier, knowing which seen blocks are dead is worth as much as knowing every seen block's future; a parameter-free rule that parks blocks seen once behind new ones recovers about half of that, and an estimated prior a little more, on workloads where the prefix points that way. Online, recency does the same job better when moving a block is free, and at the move prices we measured, under every way of billing a move we tried, none of the replayed policies that move blocks beats a static frequency heuristic on the production traces in an additive cost model, unless moves to the remote tier we could not measure cost under a quarter of a read

plus a write. Reactive replacement, including the designs built for one-hit wonders, stays within a few points of plain rules.

For whoever evaluates a memory scheduler next, the practical content is short: report the tie rule for blocks without evidence, sweep every convenience choice, bill creations as compulsory misses, and measure or bound what a move costs. The artifact implements each as a default.

ACKNOWLEDGMENTS

Use of AI-generated content. An AI assistant (Anthropic's Claude, Opus 5 and Opus 5.5 models, used through the Claude Code agentic environment) was used extensively throughout this work. It wrote most of the code in the released artifact, designed and ran most of the experiments and analyses, drafted and revised most of the text, and generated the figures from the stored results; its contributions are recorded commit by commit in the artifact's history. The "adversarial review", the audit and the re-verification passes referred to in Section 6 were internal exercises in which AI agents were prompted to act as reviewers; none was peer review, and one figure the review produced (a third LRB repair) is neither reproduced nor relied on here. Every result in this paper is re-derived from the released artifact by a checker shipped with it; a few historical figures (what an earlier version reported) come from the artifact's lab notebook. The initial concept and research idea were proposed by the author. The author takes full responsibility for the content of this paper.

REFERENCES

[1] Mustafa Canim, George A. Mihaila, Bishwaranjan Bhattacharjee, Kenneth A. Ross, and Christian A. Lang. 2010. SSD Bufferpool Extensions for Database Systems. *Proc. VLDB Endow.* 3, 1-2 (2010), 1435–1446. https://doi.org/10.14778/1920841.1921017

[2] Yihua Cheng, Yuhan Liu, et al. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv* (2025). arXiv:2510.09665

[3] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Trans. Storage* 13, 4 (2017), 1–31. https://doi.org/10.1145/3149371

[4] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, et al. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*. arXiv:2403.19708

[5] Theodore Johnson and Dennis Shasha. 1994. 2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*. 439–450. https://www.vldb.org/conf/1994/P439.PDF

[6] Anna R. Karlin, Mark S. Manasse, Larry Rudolph, and Daniel D. Sleator. 1988. Competitive Snoopy Caching. *Algorithmica* 3, 1-4 (1988), 79–119. https://doi.org/10.1007/BF01762111

[7] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. 2025. HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems. *Proc. ACM Manag. Data* 3, 4 (2025), 1–27. https://doi.org/10.1145/3749168

[8] Yiyu Liu, Minlan Yu, and Juncheng Yang. 2026. When Fancy Eviction Fails: Rethinking Cache Replacement For LLM Prefix Reuse. arXiv:2609.28870 [cs.DC]

[9] Xinyue Ma, Heelim Hong, Taegeon Um, Jongseop Lee, Seoyeong Choy, Woo-Yeon Lee, and Myeongjae Jeon. 2026. OrbitFlow: SLO-Aware Long-Context LLM Serving with Fine-Grained KV Cache Reconfiguration. *Proc. VLDB Endow.* 19, 5 (2026), 1046–1059. https://doi.org/10.14778/3796195.3796214

[10] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *2nd USENIX Conference on File and Storage Technologies (FAST)*. https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache

[11] Elizabeth J. O'Neil, Patrick E. O'Neil, and Gerhard Weikum. 1993. The LRU-K Page Replacement Algorithm for Database Disk Buffering. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. 297–306. https://doi.org/10.1145/170035.170081

[12] James Pan and Guoliang Li. 2025. Database Perspective on LLM Inference Systems. *Proc. VLDB Endow.* 18, 12 (2025), 5504–5507. https://doi.org/10.14778/3750601.3750703

[13] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*. https://www.usenix.org/conference/fast25/presentation/qin Traces: github.com/kvcache-ai/Mooncake, FAST25-release.

[14] Liana V. Rodriguez, Farzana Beente Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 341–354. https://www.usenix.org/conference/fast21/presentation/rodriguez

[15] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 529–544. https://www.usenix.org/conference/nsdi20/presentation/song

[16] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX Association, Boston, MA. https://www.usenix.org/conference/hotstorage18/presentation/vietri

[17] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *USENIX Annual Technical Conference (ATC)*. arXiv:2506.02634

[18] Daniel Lin-Kit Wong, Hao Wu, Carson Molder, Sathya Gunasekar, Jimmy Lu, Snehal Khandkar, Abhinav Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission and Prefetching for Flash Caches. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. USENIX Association, Santa Clara, CA, 347–371. https://www.usenix.org/conference/fast24/presentation/wong

[19] Chenggang Wu, Vikram Sreekanti, and Joseph M. Hellerstein. 2019. Autoscaling Tiered Cloud Storage in Anna. *Proc. VLDB Endow.* 12, 6 (2019), 624–638. https://doi.org/10.14778/3311880.3311881

[20] Haocheng Xia, William Nixon, Bintang Dwi Marthen, Pranav Bhandari, and Juncheng Yang. 2026. Learning-Augmented Heuristics: Simple Yet Smart, Robust and Interpretable Cache Eviction. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. 2241–2259. https://www.usenix.org/conference/osdi26/presentation/xia

[21] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and K. V. Rashmi. 2023. FIFO Queues are All You Need for Cache Eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 130–149. https://doi.org/10.1145/3600006.3613147

[22] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1229–1246. https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo

[23] Yingqiang Zhang, Xinjun Yang, Hao Chen, Feifei Li, Shuwen Wang, Chuan Sun, Sai Wu, and Ninglong Weng. 2026. PolarKV: Tier Locally, Serve Globally–A KV Cache over Cloud Memory and Storage. *Proc. VLDB Endow.* 19, 12 (2026), 4480–4493. https://doi.org/10.14778/3827998.3828047

[24] Xinjing Zhou, Joy Arulraj, Andrew Pavlo, and David Cohen. 2021. Spitfire: A Three-Tier Buffer Manager for Volatile and Non-Volatile Memory. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. 2195–2207. https://doi.org/10.1145/3448016.3452819